
Jiaxing Wu
2023533160
wujx2023@shanghaitech.edu.cn

Xinyu Jia
2023533102
jiaxy2023@shanghaitech.edu.cn

Abstract

This report summarizes four acoustic communication projects that together form an end-to-end audio-based networking system: Project 1 (Physical Layer), Project 2 (Medium Access Control), Project 3 (Aethernet IP Networking), and Project 4 (Application Layer). We start from basic audio device verification and BPSK modulation, extend to reliable and fair multi-node MAC protocols, build a full IP stack with routing and NAT, and finally implement TCP/HTTP protocols to enable real-world web browsing over sound waves.

1 Project Overview

This report summarizes the implementation of three major acoustic communication projects:

- **Project 1:** Acoustic physical layer — two-tone sanity test and a BPSK modem with chirp preamble.
- **Project 2:** Medium Access Control (MAC) — cable PHY, ACK protocol, CSMA, and multi-node extension.
- **Project 3:** *Aethernet* — an audio-based IP networking stack with routing, NAT, fragmentation, and ARP proxy.
- **Project 4:** *Above IP* — Implementation of DNS resolution, a custom TCP stack, HTTP client, and transparent gateway for web browsing.

Together, these projects build an end-to-end audio-based communication system capable of IP routing, NAT, fragmentation, and full-duplex multi-node communication over sound.

2 Project 1: Physical Layer Communication

2.1 Two-Tone Diagnostic Test

The system first verifies the JUCE audio callback pipeline by playing the superposition of two sine waves: a 1 kHz tone and a 10 kHz tone at a sampling rate of 44.1 kHz. This two-tone test validates:

- Output device initialization,
- Real-time callback correctness,
- Continuous synthesis and streaming of sample buffers.

Successful playback confirms that the audio pipeline is functioning correctly before deploying more complex modulation schemes.

2.2 BPSK Audio Modem with Chirp Preamble

The main physical layer (PHY) modem in Project 1 implements binary phase-shift keying (BPSK) over an audio channel.

- Sampling rate: 44 100 Hz.
- Carrier frequency: 10 kHz.
- Modulation: BPSK, where bit ‘1’ is mapped to $+\sin(2\pi f_{\text{car}}t)$ and bit ‘0’ to $-\sin(2\pi f_{\text{car}}t)$.
- Samples per bit: 44 (raw rate ≈ 1 kbps).
- Preamble: linear up-chirp from 2 kHz to 10 kHz followed by down-chirp from 10 kHz to 2 kHz.
- Error detection: CRC-8 per frame.

Transmitter. The transmitter reads bits from `INPUT.txt`, groups them into payloads, appends a CRC-8 checksum, and constructs frames with the structure:

$$\text{Frame} = \text{Preamble} \parallel \text{Payload (100 bits)} \parallel \text{CRC-8 (8 bits)}.$$

Each bit is mapped to BPSK symbols at 10 kHz, and the resulting audio samples are saved as `OUTPUT.wav` and optionally played back.

Receiver. The receiver continuously records from the microphone and applies:

1. **DC blocking** using a single-pole filter $y[n] = x[n] - x[n-1] + a y[n-1]$.
2. **Preamble detection** via normalized correlation with a local chirp template in a trailing window.
3. **Polarity detection** from the sign of the correlation peak to handle possible π phase flips.
4. **Bit decisions** by multiplying with a one-bit carrier template and integrating over a mid-bit guard interval; positive sum $\Rightarrow 1$, negative sum $\Rightarrow 0$.
5. **CRC checking** of the decoded bits; only frames with valid CRC contribute payload bits to `OUTPUT.txt`.

This establishes a robust, low-rate, point-to-point audio PHY link that later projects will build upon.

3 Project 2: Medium Access Control

Project 2 extends the physical-layer modem into a full MAC protocol that supports higher throughput, acknowledgements, retransmissions, and multiple nodes sharing the channel.

3.1 High-Speed Cable Connection (Task 1)

For a clean electrical connection over audio cable, we implement a Manchester coded PHY at 48 kHz:

- Each “chip” (half-bit) uses 2 samples; each full bit uses 4 samples.
- Raw data rate:

$$R = \frac{48000 \text{ samples/s}}{4 \text{ samples/bit}} = 12000 \text{ bps.}$$

- The 6250 byte file (50 000 bit) is split into 500 frames of 100 bits.

Each frame contains:

- **Preamble:** A 240-sample (5 ms) linear chirp from 4 kHz to 12 kHz for synchronization.
- **Payload:** 100 data bits (Manchester-coded).
- **CRC:** 8 bits using polynomial `0x1A7`.
- **Gap:** 120 samples (2.5 ms) of silence to separate frames.

The total frame length is $240 + 432 + 120 = 792$ samples, giving a frame duration of:

$$T_{\text{frame}} = \frac{792}{48000} \approx 0.0165 \text{ s,}$$

and a total transmission time

$$500 \times 0.0165 \approx 8.25 \text{ s},$$

which satisfies the requirement of finishing within 10 s.

The receiver uses a matched filter for the chirp preamble, then decodes the Manchester data and checks CRC before adding payload bits to a buffer. Once all frames are received, bits are packed into bytes and written to `OUTPUT.bin`.

3.2 Stop-and-Wait ACK Protocol (Task 2)

To guarantee reliable delivery in the presence of errors, we implement a Stop-and-Wait ACK protocol on top of the PHY. A 16-bit MAC header is added:

Field	Bits
Destination address	3
Source address	3
Frame type (DATA/ACK)	1
Frame ID	9

The frame types are:

- **DATA frame (NODE1→NODE2):**
 - MAC header: 16 bits.
 - MAC payload: 100 bits (file data).
 - CRC: 8 bits.
 - Total PHY bits: 124 bits.
- **ACK frame (NODE2→NODE1):**
 - MAC header: 16 bits (includes the acknowledged Frame ID).
 - MAC payload: 0 bits.
 - CRC: 8 bits.
 - Total PHY bits: 24 bits.

Sender state machine. The sender implements a Stop-and-Wait protocol with states such as: `IDLE` and `WAITING_FOR_ACK`. It:

1. Sends a DATA frame for the current frame index.
2. Starts a timeout timer (e.g., 300 ms).
3. If a valid ACK with matching Frame ID is received before timeout:
 - Stop the timer,
 - Advance to the next frame.
4. If timeout expires:
 - Retransmit the same DATA frame.
 - Abort after a maximum number of retransmissions.

Receiver behavior. The receiver listens for DATA frames, checks the CRC, and if valid:

- Stores the 100-bit payload,
- Immediately prepares and transmits an ACK frame back to the sender.

This guarantees 100% data accuracy as long as corrupted frames are eventually retransmitted successfully.

3.3 CSMA with Listen-Before-Talk (Task 3)

To enable multiple nodes to share the same medium, we implement a CSMA (Carrier Sense Multiple Access) protocol on top of the Stop-and-Wait ACK mechanism.

Carrier sensing. Each node continuously computes the average signal power in the incoming audio stream. A boolean flag `isChannelBusy` is set according to a *silence threshold*; if the power exceeds the threshold, the channel is considered busy.

Listen-Before-Talk (LBT). Before sending either a DATA or ACK frame:

- If the channel is idle, the node transmits immediately.
- If the channel is busy, the node defers, enters a `WAITING_FOR_CHANNEL` state, and starts a short random backoff timer.

Collision handling and Binary Exponential Backoff. When an ACK is not received within the timeout, the sender treats it as a collision and:

1. Increments the retransmission count k .
2. Computes $n = \min(k, \text{MAX_BACKOFF_EXPONENT})$.
3. Randomly selects a slot $r \in [0, 2^n - 1]$.
4. Sets backoff time $T_{\text{backoff}} = r \cdot T_{\text{slot}}$.

After backoff, the node retries `sendDataFrame` with the same Frame ID.

Fairness: “winner yields”. To avoid channel capture, a node that successfully sends a frame and receives its ACK *does not* immediately send the next frame. Instead, it resets the retransmission count and voluntarily enters a random backoff before its next DATA frame, giving other nodes a fair chance to access the channel.

3.4 Scaling to Four Nodes (Task 6)

For the 4-node scenario, two major optimizations are added:

3.4.1 Go-Back-N Sliding Window

Instead of pure Stop-and-Wait, the sender uses a Go-Back-N (GBN) sliding window:

- Window size $W = 10$ frames.
- The sender transmits up to W frames in a *burst* without waiting for ACKs after each frame.
- The receiver buffers frames within the window and sends a *cumulative ACK* after the last expected frame in the window.
- If the burst timeout expires, the sender “goes back” and retransmits the entire window starting from the last confirmed frame.

3.4.2 Coordinated Access (Token-like Handover)

To reduce collisions among four active nodes, a coordination mechanism is superimposed on CSMA:

- One node is designated as the initial “master” and allowed to send the first burst.
- Once a burst is successfully transmitted and acknowledged, the sender does not continue immediately. Instead, it sends a special “handover” frame (with a reserved Frame ID) to the next node in a predefined order.
- A node that receives this handover frame sets a local flag indicating “it is my turn to send”, then after a random CSMA backoff starts its own burst.

The combination of GBN, CSMA, and coordinated handover ensures:

- All four nodes complete their transfers within 120 s,
- The difference between completion times $\max(T_i) - \min(T_i)$ is less than 15 s, satisfying the fairness requirement.

4 Project 3: Aethernet — IP over Audio

Project 3, named *Aethernet*, builds a complete IP networking stack on top of the audio-based PHY/MAC, enabling air-gapped devices to access the Internet through a software router.

4.1 ICMP Echo over Audio (Audio Ping)

The first step is implementing raw IP and ICMP (ping) over the audio link:

- Custom `IPHeader` and `ICMPHeader` structures are defined, including checksum calculation.
- IP packets are encapsulated into Aethernet frames and transmitted via the audio modem.
- On reception, the system reconstructs IP packets and replies to ICMP Echo Requests.

To keep audio playback stable, heavy packet processing is offloaded from the real-time audio callback thread using asynchronous message calls.

4.2 Software Router with Static Routing

NODE2 acts as a software router between three interfaces:

- Audio (Aethernet) interface,
- Wi-Fi interface (connected to the campus LAN/Internet),
- Hotspot interface (for NODE3, e.g., a smartphone).

Using `libpcap`, dedicated capture threads read and inject packets on Wi-Fi and hotspot adapters. Packets destined for the Aethernet subnet (e.g., `172.18.x.x`) are forwarded into the audio modem; others are forwarded between Wi-Fi and hotspot according to static routing policies.

4.3 Network Address Translation (NAT)

To allow internal audio-node hosts (e.g., NODE1 at `172.18.0.1`) to access the Internet, the router implements NAT:

- **SNAT (Source NAT):** On outbound packets, the source IP `172.18.0.1` is replaced by NODE2's Wi-Fi IP.
- **DNAT (Destination NAT):** On inbound replies, the router uses a NAT table keyed by ICMP ID (and possibly other fields) to restore the original internal destination IP.

An optional NAT traversal feature inspects the ICMP payload. If it contains a specific internal IP address (e.g., `172.18.0.1`), the packet is routed into the audio network, enabling an external host to reach NODE1.

4.4 IP Fragmentation and Reassembly

Because the Aethernet MTU is limited (e.g., 196 bytes), large IP packets must be fragmented:

- On send, the system slices packets into fragments, duplicating the IP header and correctly setting the MF (More Fragments) flag and fragment offset fields.
- On receive, fragments are stored in a reassembly buffer indexed by (ID, source, destination). When all fragments arrive and MF=0 is observed, the full packet is reconstructed.
- Only fully reassembled packets are forwarded to the OS or router logic.

A small rate limit (sleep) is applied when sending many fragments to avoid overwhelming the receiver's buffer.

4.5 Virtual Network Device Bridge

To integrate Aethernet into the OS networking stack, a virtual network device is used on NODE1 (e.g., the Microsoft KM-TEST Loopback Adapter):

- The loopback adapter is assigned an IP address `172.18.0.1/24`.
- A bridge process captures packets from this adapter via `libpcap`, encapsulates them into Aethernet frames, fragments if necessary, and sends them via audio.
- Conversely, reassembled IP packets received via audio are injected back into the loopback adapter using `pcap_sendpacket`.

From the OS perspective, Aethernet becomes a normal IP interface. Users can run commands such as:

```
ping 172.18.0.2      % ping NODE2 over audio
ping 1.1.1.1         % ping Internet through NODE2
```

4.6 Proxy ARP

Since there is no real Ethernet broadcast domain, ARP resolution must be emulated. NODE1 implements Proxy ARP:

- It listens for ARP Requests on the loopback adapter (e.g., for `172.18.0.2`).
- It responds with a fake MAC address (e.g., `02:00:AA:BB:CC:DD`).
- The OS then sends IP packets to that MAC, which actually belong to the Aethernet bridge.

This allows standard TCP/IP applications to work on top of the audio link without modifications.

5 Project 4: Application Layer

Project 4 moves "Above IP" to implement standard Internet protocols, enabling NODE1 to browse the web.

5.1 DNS Resolution (Task 1)

We implemented a DNS client on NODE1. When a user queries a domain (e.g., `baidu.com`):

- NODE1 constructs a standard DNS Query UDP packet.
- The packet is routed via audio to NODE2.
- NODE2 acts as a NAT router, forwarding the query to a real DNS server (e.g., `223.5.5.5`) via the Internet.
- The response is relayed back to NODE1, which parses the IP address.

5.2 HTTP Client and Custom TCP Stack (Task 2)

To support HTTP, we implemented a lightweight TCP stack on NODE1. This includes:

- **State Machine:** Handling states like `TCP_HANDSHAKE`, `TCP_ESTABLISHED`, and `TCP_FIN_WAIT`.
- **Three-Way Handshake:** NODE1 sends SYN, waits for SYN+ACK, and sends ACK.
- **Sequence Number:** We customized the Initial Sequence Number (ISN) to satisfy the specific requirement: $0xH_1...H_8$ where $H_i \in \{1..8\}$ and distinct. We used `0x12345678`, verified via Wireshark on NODE2.
- **HTTP Request:** After connection, NODE1 constructs a valid GET request string and parses the HTML response.

6 Conclusion

Across Projects 1–4, we have built a complete communication stack over audio:

- **Physical layer:** BPSK modulation with chirp preambles.
- **MAC layer:** Stop-and-Wait, CSMA, and Go-Back-N.
- **Network layer:** IP routing, NAT, Fragmentation, and Proxy ARP.
- **Application layer:** DNS resolution, TCP state management, and HTTP web browsing.

The final system supports bidirectional real-time data transmission and allows a standard computer to access the modern Internet solely through sound waves, demonstrating the power of software-defined networking.